

Programacion funcional

Tabla de contenidos

1	Describiendo el problema, no el procedimiento	3
1.1	Del estado a la transformación	3
1.2	Composición de funciones	4
1.3	Funciones puras	4
1.4	Inmutabilidad	5
1.5	¿Qué aporta realmente la programación funcional?	5
1.6	Un nuevo nivel de abstracción	5

1 Describiendo el problema, no el procedimiento

Los lenguajes de tercera generación (3GL) supusieron un enorme avance en la construcción de software. El ingeniero describía, paso a paso, el algoritmo que debía ejecutar el ordenador, sin embargo, conforme los sistemas crecían en tamaño y complejidad, comenzó a surgir una pregunta cada vez más importante.

¿Es realmente necesario describir todos los detalles del procedimiento?

En muchos programas, una parte significativa del código no representa el problema que se desea resolver, sino la forma en que el ordenador debe recorrer estructuras de datos, mantener estados temporales o controlar el flujo de ejecución, estas tareas son necesarias para la máquina, pero rara vez forman parte del conocimiento del dominio del problema.

La programación funcional propone un cambio de perspectiva: En lugar de describir detalladamente cómo realizar cada paso del algoritmo, el desarrollador expresa qué transformación desea aplicar sobre los datos, el interés deja de centrarse en el procedimiento y pasa a centrarse en el resultado de cada transformación.

1.1. Del estado a la transformación

En la programación imperativa el estado del programa cambia continuamente.

Por ejemplo:

```
int total = 0  
  
for (i in ventas) total += i;
```

El algoritmo mantiene una variable cuyo valor cambia en cada iteración, desde el punto de vista del procesador resulta perfectamente válido sin embargo, desde el punto de vista del dominio del problema, esa variable intermedia apenas aporta información. En programación funcional el mismo problema puede expresarse de una forma mucho más cercana a la intención del desarrollador.

```
total <- sum(ventas)
```

El desarrollador ya no explica cómo recorrer la colección si no que simplemente indica cuál es el resultado que desea obtener, es el propio lenguaje quien decide cómo ejecutar esa operación.

1.2. Composición de funciones

Una de las características más importantes de la programación funcional consiste en considerar las funciones como elementos que pueden componerse entre sí:

1. Cada función recibe unos datos.
2. Produce una transformación.
3. Entrega un resultado que puede inyectarse a la siguiente función.

Por ejemplo, utilizando el ecosistema **tidyverse** de R:

```
ventas_validas <-  
  ventas |>  
  filter(importe > 0) |>  
  mutate(iva = importe * 0.21) |>  
  arrange(desc(importe))
```

Cada línea representa una transformación independiente, no aparecen índices, no existen bucles explícitos, las variables temporales prácticamente desaparecen. El código deja de describir el recorrido y comienza a describir una secuencia lógica de transformaciones, esta forma de trabajar facilita enormemente la lectura del programa por que cada operación expresa claramente su intención.

1.3. Funciones puras

Otro de los pilares fundamentales de la programación funcional son las funciones puras: Una función pura siempre produce el mismo resultado cuando recibe los mismos parámetros.

- No modifica variables globales.
- No altera el estado del sistema.
- No produce efectos secundarios.

Por ejemplo:

```
precio_con_iva <- function(precio) {  
  precio * 1.21  
}
```

Su comportamiento es completamente predecible. Esta propiedad simplifica las pruebas, facilita la depuración y permite razonar sobre el software con mucha mayor facilidad.

1.4. Inmutabilidad

En programación funcional los datos tienden a considerarse inmutables, en lugar de modificar un objeto existente, se genera uno nuevo que representa el resultado de la transformación. Aunque internamente el lenguaje pueda optimizar este proceso, conceptualmente el programa deja de construirse mediante cambios continuos de estado y pasa a entenderse como una cadena de transformaciones sucesivas. Esta filosofía reduce numerosos errores relacionados con estados compartidos, concurrencia y efectos inesperados.

1.5. ¿Qué aporta realmente la programación funcional?

La programación funcional no elimina la necesidad de diseñar algoritmos, el ingeniero continúa siendo responsable de definir la lógica que resuelve el problema sin embargo, desplaza parte de la responsabilidad desde el desarrollador hacia el propio lenguaje.

El programador deja de describir numerosos mecanismos de bajo nivel y comienza a expresar transformaciones de un nivel mucho más cercano al problema que desea resolver, el resultado son programas más compactos, más expresivos y, en muchos casos, más fáciles de mantener.

1.6. Un nuevo nivel de abstracción

La evolución de los lenguajes de programación puede interpretarse como una búsqueda constante de niveles superiores de abstracción, en los lenguajes imperativos, el desarrollador describe cada paso del algoritmo, en la programación funcional, describe transformaciones sobre los datos y delega en el lenguaje muchos de los detalles de ejecución aunque el ordenador continúa ejecutando instrucciones.

Pero el ingeniero comienza a expresarse utilizando conceptos cada vez más próximos al problema que desea resolver y cada vez más alejados del funcionamiento interno de la máquina, Este cambio representa un paso más en la evolución de la Ingeniería del Software y, aunque todavía seguimos escribiendo algoritmos, comienza a apreciarse una tendencia que reaparecerá con mucha más fuerza décadas después.

Cada nueva generación de herramientas permite al ingeniero describir **qué desea conseguir**, mientras delega progresivamente **cómo conseguirlo**. La programación funcional constituye uno de los primeros pasos importantes en esa dirección.

En los capítulos siguientes veremos cómo esa tendencia continúa con los lenguajes declarativos y culmina, al menos por el momento, con los Large Language Models, donde el desarrollador comienza a expresar objetivos e intenciones más que algoritmos detallados.